



Design and Implementation of a Session-Managed Multiplayer Tic Tac Toe Web Application Using the Spring Framework

Prepared by:

Ezebuiro AbbaJustice

Supervisor:

Vegera Jozsef

Academic Year 2025/2026

Abstract

This thesis presents the design, implementation, and evaluation of a distributed multiplayer Tic Tac Toe web application built on the Java Spring ecosystem. The core problem addressed is how to maintain consistent and authoritative game state across multiple clients in a turn-based system operating over the inherently stateless HTTP protocol, while ensuring correctness under concurrent access and secure user interaction.

The system resolves this through three coordinated architectural mechanisms: server-side state persistence using PostgreSQL as the single source of truth, transactional enforcement of game rules via Spring's `@Transactional` service layer to prevent race conditions, and a short-polling synchronisation model enabling clients to observe state changes without maintaining persistent connections. Authentication and request integrity are handled through Spring Security using session-based authentication, with CSRF protection configured according to application requirements.

Evaluation was conducted through a suite of 13 automated tests — including unit tests for game logic and integration tests for the security filter chain — alongside live LAN-based multiplayer testing. Results demonstrate that transactional boundaries effectively prevent concurrent move corruption, that polling at one-second intervals provides sufficient responsiveness for turn-based interaction, and that the system maintains consistent game progression across distributed clients. The Minimax-based AI opponent performs optimally at maximum search depth, providing a baseline for automated gameplay.

Keywords: Spring Boot 3, Spring Security, BCrypt, Thymeleaf, PostgreSQL, Multiplayer, Short-Polling, Minimax, MVC, Java 17

Table of Contents

Contents

Abstract 2

Table of Contents 3

1. Introduction 6

 1.1 Context and Motivation..... 6

 1.2 Research Question and Objectives 7

 1.3 Scope and Limitations..... 8

 1.4 Document Structure 8

2. Literature Review 9

 2.1 The Stateless Web and Session Management 9

 2.2 Web Application Architecture: MVC and Layered Design.....10

 2.3 Security: Authentication and Password Storage.....11

 2.4 Artificial Intelligence: The Minimax Algorithm.....11

 2.5 Glossary of Key Terms13

3. Development Tools and Environment.....14

 3.1 Backend Technologies14

 3.2 Security Layer14

 3.3 Frontend Technologies.....15

 3.4 Database.....16

 3.5 Testing Tools.....16

 3.6 Development Hardware.....16

4. Program Requirements18

 4.1 Functional Requirements.....18

 4.2 Non-Functional Requirements.....19

5. Use Cases, User Stories, and Acceptance Criteria20

 Use Case Diagram20

 5.1 Use Case Summary.....21

 5.2 User Stories22

 US-01: Secure Registration and Login22

 US-02: Creating and Joining a PvP Game22

 US-03: Real-Time Board Synchronisation.....22

 US-04: Turn Integrity and Cheat Prevention23

 US-05: AI Opponent with Difficulty Selection.....23

 US-06: Undo in CPU Mode.....23

 5.3 Acceptance Criteria24

6. System Design25

 6.1 User Roles.....25

 6.2 Usage Scenarios25

Scenario A: PvP Match	25
Scenario B: CPU Match (IMPOSSIBLE difficulty)	25
6.3 Database Schema	26
7. User Interface Design	28
7.1 Page Inventory	28
7.2 Mode Selection Page	28
7.3 Game Board Page	29
7.4 Game-Over Overlay	30
8. Final Data Model and System Architecture	31
8.1 Entity Model	31
AppUser Entity	31
Game Entity (selected fields)	31
8.2 Layered Architecture	32
9. Multiplayer Synchronisation Model	33
9.1 The Design Decision: Polling vs. WebSocket	33
9.2 Polling Implementation	33
9.3 Synchronisation Flow	34
9.4 Server-Side Timer Implementation	34
10. Security Implementation	37
10.1 Spring Security Filter Chain	37
10.2 Password Storage: BCrypt	38
10.3 BCrypt Migration for Legacy Accounts	38
10.4 CSRF Handling	40
10.5 Server-Side Turn Enforcement	40
11. AI Opponent: Minimax with Difficulty Levels	41
11.1 Algorithm Overview	41
11.2 Implementation	41
11.3 Difficulty Levels	42
12. Testing and Evaluation	43
12.1 Testing Strategy	43
12.2 Unit Tests: GameTest	43
12.3 Integration Tests: TictactoeApplicationTests	44
12.4 Manual LAN Testing	44
12.5 Known Limitations	47
13. Analysis	48
13.1 Architectural Trade-offs and System Behaviour	48
13.2 Observations on Scalability and Design Constraints	49
14. Conclusion and Future Work	50
14.1 Conclusion	50

14.2 Future Work50

References.....51

Appendix: Project Repository53

1. Introduction

1.1 Context and Motivation

The fundamental nature of HTTP is stateless: each request is processed in isolation, with no inherent memory of what preceded it. This creates a genuine engineering problem when building interactive, multi-user applications where shared state, such as a game board mid-match, must remain consistent across clients that are physically separated and communicating asynchronously.

Tic Tac Toe was chosen as the application domain deliberately. Its rule set is small enough to implement completely from scratch in a single academic project, yet the requirements for turn-based integrity, concurrent access control, and real-time synchronisation mirror those of much larger distributed systems. The goal was not to build the most technically impressive game possible, but to use a simple domain to explore and demonstrate non-trivial backend engineering decisions: how do you prevent two players from moving simultaneously? How do you keep two browser windows in sync without a persistent connection? How do you ensure a player cannot manipulate another's session?

This project uses '**distributed**' to describe an architecture where multiple independent client sessions on physically separate devices coordinate game state through a central server, addressing the synchronization and consistency challenges inherent in stateless HTTP communication.

These questions are explored using the Java Spring ecosystem — specifically Spring Boot 3, Spring Security, Spring Data JPA, and the Thymeleaf template engine. This stack was selected both for its relevance to enterprise Java development and for the depth of its documentation, which allowed implementation decisions to be grounded in established conventions rather than ad hoc solutions.

1.2 Research Question and Objectives

Research Question:

How can a session-managed, turn-based multiplayer game maintain consistent and authoritative state across distributed clients communicating via stateless HTTP interactions, and what architectural trade-offs does this impose in terms of security, state synchronisation, and concurrent access control?

This question is addressed through the following concrete objectives:

1. Implement a complete, rules-compliant Tic Tac Toe game engine in Java with win, draw, and undo detection.
2. Build a secure multi-user authentication system using Spring Security with BCrypt password hashing.
3. Design and implement a REST-based short-polling synchronisation mechanism to keep distributed clients consistent without WebSocket overhead.
4. Ensure transactional integrity for concurrent move submissions using Spring's `@Transactional` annotation backed by PostgreSQL.
5. Implement an AI opponent using the Minimax algorithm at three selectable difficulty levels.
6. Validate the system through automated unit and integration tests.

1.3 Scope and Limitations

The scope of this project is limited to a server-rendered web application deployed within a local area network (LAN) environment. The system supports two game modes: Player vs Player (PvP), where two authenticated users interact via a polling-based synchronisation mechanism, and Player vs CPU, where a single player competes against a Minimax-based AI. The application does not support real-time push notifications, WebSocket communication, or public internet deployment; these are identified as potential extensions in Chapter 14.

The board size is fixed at 3×3 . The Minimax evaluation logic and win-detection mechanisms are implemented specifically for this configuration, which forms the basis for all testing and documentation. Extension to larger board sizes is identified as a future enhancement.

1.4 Document Structure

This thesis is structured to progress from theoretical foundations through system design and implementation to evaluation and reflection.

Chapter 2 reviews relevant academic and technical literature, establishing the conceptual background for REST constraints, synchronisation models, and web application architecture. Chapters 3 to 5 define the development context and system requirements, including tools, functional specifications, and user-oriented design artefacts.

Chapters 6 through 11 present the system design and implementation. This includes database schema design, application architecture, synchronisation mechanisms, security considerations, and the implementation of the AI opponent. Particular emphasis is placed on how architectural decisions address consistency, concurrency, and communication constraints.

Chapter 12 evaluates the system through testing, including known limitations. Chapter 13 analyses architectural trade-offs and directly addresses the research question. Chapter 14 presents the conclusion and outlines directions for future work.

2. Literature Review

2.1 The Stateless Web and Session Management

Fielding (2000) formalised the REST architectural style in his doctoral dissertation, identifying statelessness as one of its defining constraints. In a REST-compliant system, each client request must contain all information necessary for the server to understand and process it independently, without relying on hidden conversational state stored between requests. Richardson (2019) discusses how these principles inform modern service-based and microservice architectures, particularly in distributed environments where scalability and decoupling are critical.

Strict adherence to REST discourages the use of server-side session state, as it introduces dependency on stored context between requests. However, in practice, many web applications employ session-based mechanisms for authentication and interaction management. This project adopts Spring Security's session-based authentication model, in which a `JSESSIONID` cookie is issued upon successful login and included in subsequent requests. The server maintains session data associated with this identifier, including authentication context and CSRF protection tokens. While this approach departs from strict REST statelessness, it reflects widely adopted web application practices and prioritises security and implementation clarity. Application state, such as game progression, is persisted independently as a resource, while HTTP session state is used solely for authentication and request validation.

Grigorik (2013) discusses client–server communication constraints in web applications, noting that HTTP does not allow the server to initiate communication and therefore requires techniques such as polling or streaming-based transports. He describes polling as a simple approach that introduces a trade-off between latency and network overhead, where shorter intervals increase responsiveness but generate unnecessary traffic, while longer intervals reduce overhead at the cost of delayed updates. Pimentel and Nickerson (2012) evaluate the WebSocket protocol as a bidirectional, low-latency communication mechanism, demonstrating its efficiency advantages over HTTP polling and long-polling in high-frequency update scenarios.

Taken together, these works define a clear trade-off between implementation simplicity and communication efficiency in web-based client–server interaction. Polling-based approaches

offer simplicity at the cost of increased latency and network overhead, while persistent connection mechanisms such as WebSocket provide lower-latency communication with additional implementation complexity. The implications of these trade-offs for system design are examined in later chapters, where the chosen synchronisation model is evaluated in the context of the application's requirements.

2.2 Web Application Architecture: MVC and Layered Design

The Model–View–Controller (MVC) pattern, originally introduced by Reenskaug (1979), was later adapted for web application development as a means of separating user interface concerns from application logic. Fowler (2002) situates MVC within a broader set of enterprise application architecture patterns, emphasising the separation of presentation, domain logic, and data access responsibilities, and identifying this separation as a fundamental principle of maintainable software design.

The architecture of this project follows this separation through the use of the Spring MVC framework. Request handling is structured around the Front Controller pattern, implemented by Spring's `DispatcherServlet`, which provides a centralised request-processing mechanism and delegates handling to configurable components such as controllers (Spring Security Team, 2024). Within this structure, controller classes manage HTTP request–response interactions, service-layer components encapsulate domain logic, and repository interfaces abstract persistence. This layered organisation structures dependencies from presentation to domain and persistence layers, supporting modular design and separation of concerns.

The design is further guided by the Single Responsibility Principle, as articulated by Martin (2008). In this project, responsibilities are explicitly partitioned across components: the Game entity encapsulates game state and rule invariants, the GameService acts as a service layer coordinating game logic and AI decision-making, and the GameController handles HTTP interactions. This separation aligns with Fowler's observation that non-visual components are easier to test, and enables the Minimax implementation within the service layer to be unit-tested independently of the web and persistence layers.

2.3 Security: Authentication and Password Storage

Spring Security provides a comprehensive framework for authentication and authorisation within Spring Boot applications (Spring Security Team, 2024). It implements a filter chain architecture in which each incoming HTTP request passes through a configurable sequence of security filters before reaching the application's controllers. In this project, these filters handle form-based authentication and session management.

Secure password storage is a fundamental requirement for web applications. The OWASP Foundation's *Password Storage Cheat Sheet* (OWASP Foundation, 2023) identifies `bcrypt` as an appropriate password hashing function due to its adaptive design, which increases resistance to brute-force and offline attacks.

The initial implementation of this project used Spring Security's `NoOpPasswordEncoder`, which stores passwords in plaintext, as a development shortcut. This was replaced with `BCryptPasswordEncoder` during the security hardening phase, aligning the implementation with OWASP guidance and eliminating a critical vulnerability present in earlier iterations.

2.4 Artificial Intelligence: The Minimax Algorithm

The theoretical foundation for analysing adversarial decision-making was established by von Neumann and Morgenstern (1944) through the formalisation of zero-sum game theory. A two-player, turn-based game with perfect information—such as Tic Tac Toe—serves as a canonical example: one player's gain is precisely the other's loss, and both participants maintain complete knowledge of the game state at all times. According to Zermelo's Theorem (1913), such games are strictly determined, meaning an optimal strategy exists that allows a player to force a specific outcome.

In this project, the Minimax algorithm provides a computational method for determining optimal play in such games. Russell and Norvig (2020) provide the standard formulation of the algorithm, which constructs a search tree of possible future states. At each level of the tree, the maximising player selects the move with the highest evaluated outcome, while the minimising player is assumed to act optimally by selecting the lowest. For a 3×3 Tic Tac Toe grid, the state space is bounded by 9! (362,880) possible sequences, making an exhaustive search computationally trivial on modern hardware.

To improve efficiency, this implementation incorporates alpha–beta pruning. As described by Russell and Norvig (2020), this technique eliminates branches of the search tree that cannot influence the final decision, reducing the number of evaluated nodes without affecting correctness.

Finally, the software architecture ensures this logic is decoupled from the wider system. Following Bloch’s (2018) principles in *Effective Java*, the Minimax logic is encapsulated within dedicated, stateless methods. This helps ensure the algorithm operates on a simple board representation without side effects, facilitating unit testing and maintaining independence from the user interface and persistence layers.

2.5 Glossary of Key Terms

The following terms appear throughout this document and are defined here for clarity.

Table I - Key Terms

Term	Definition as used in this project
Short-polling	A client-side pattern where the browser repeatedly sends HTTP GET requests at a fixed interval (1 second) to check for updated server state.
Session	A server-side data structure, identified by a cookie in the browser, that persists the authenticated user's identity across multiple HTTP requests.
@Transactional	A Spring annotation that wraps a method in a database transaction, ensuring all database operations within it succeed or fail atomically.
Minimax	A recursive algorithm that evaluates game states by simulating all future moves and selecting the option that maximises the AI's outcome.
BCrypt	A password hashing function that incorporates a salt and configurable cost factor to resist brute-force attacks.
Principal	In Spring Security, the object representing the currently authenticated user, accessible from any controller method via the Principal parameter.
MVC	Model-View-Controller: an architectural pattern separating data (Model), rendering (View), and request handling (Controller).
ACID	Atomicity, Consistency, Isolation, Durability: the properties that database transactions must satisfy to maintain data integrity.
Race condition	A bug where the outcome of a computation depends on the relative timing of two or more concurrent operations.
ElementCollection	A JPA annotation mapping a collection of value types (e.g. List<String>) to a separate database table linked by a foreign key.

3. Development Tools and Environment

This chapter describes the specific software and hardware configuration used during development and testing. The focus is on the tools as they were actually used, not as general-purpose descriptions.

3.1 Backend Technologies

The application is authored in Java 17 (LTS), selected for its long-term support and compatibility with the Spring Boot 3.x ecosystem. The project leverages Spring Boot 3.5.5, utilising auto-configuration to reduce manual configuration of components such as the embedded servlet container, JSON serialisation, and ORM integration.

Spring Data JPA, with Hibernate as the ORM provider, maps the `AppUser` and `Game` entities to PostgreSQL tables. During development, the `spring.jpa.hibernate.ddl-auto` property is set to `update`, enabling Hibernate to adjust the database schema as entity models evolve. Application settings are centralised within the `application.properties` file, with no reliance on XML-based configuration.

Maven serves as the build automation and dependency management tool. The project's `pom.xml` defines core dependencies, including Spring Boot starters and the Lombok library, which is used to minimise boilerplate code in the domain model through annotations such as `@Data`.

Spring Boot's auto-configuration mechanism reduces manual setup by automatically configuring components based on classpath dependencies and application context (VMware Tanzu, 2024).

3.2 Security Layer

Spring Security 6.x is integrated into the application via the `spring-boot-starter-security` dependency, providing authentication, authorisation, and request-level protection within the Spring Boot ecosystem.

Security configuration is defined in the `SecurityConfig` class using the Lambda DSL introduced in recent versions of Spring Security. The application enforces authentication for protected endpoints while allowing unauthenticated access to specific routes such as

`/login`, `/register`, static resources, and error handling endpoints. All `/game/**` routes require an authenticated session.

Authentication is session-based. Upon successful login, a session cookie is issued and used to identify the user in subsequent requests, enabling consistent identity management across multiple HTTP interactions.

Password storage is handled using Spring Security's `BCryptPasswordEncoder`, which applies salted hashing to protect user credentials and replaces earlier plaintext storage approaches.

Cross-Site Request Forgery (CSRF) protection is enabled by default, with selective configuration applied to specific endpoints as required by the application's request handling model.

The detailed configuration of the security filter chain, password encoding strategy, CSRF handling, and server-side validation mechanisms are described in Chapter 10.

3.3 Frontend Technologies

Thymeleaf 3.x is used as the server-side template engine to render HTML views. It enables dynamic HTML generation through template expressions and model binding (Thymeleaf Project, 2023). It was selected over client-side frameworks such as React or Vue because the application primarily relies on server-rendered content at page load, with only incremental updates required during gameplay. Dynamic behaviour is handled through JavaScript, allowing the frontend to remain lightweight without introducing the build complexity of a single-page application.

Client-side logic is implemented in plain ES6 JavaScript using the browser's native Fetch API, which provides a promise-based interface for asynchronous HTTP requests (Mozilla Developer Network, 2024). No external frontend libraries (e.g. jQuery or Axios) are used. The polling mechanism, dynamic board rendering, and game-over overlay logic are implemented directly within the `game.html` template. The polling-based synchronisation model is described in detail in Chapter 9.

3.4 Database

PostgreSQL 16 is used as the primary database for persistent storage. During development and automated testing, an H2 in-memory database is used to enable fast iteration and isolated test execution. The behaviour and configuration of PostgreSQL in this project follow the official PostgreSQL 16 documentation (PostgreSQL Global Development Group, 2024), ensuring alignment with supported transactional and storage mechanisms.

The switch between H2 and PostgreSQL is managed via Spring profiles configured in the `application.properties` file. The H2 console is enabled in development, which requires disabling frame options in the security configuration. The database schema is managed automatically through Hibernate's schema update mechanism during development.

3.5 Testing Tools

JUnit 5 is used for both unit and integration testing. The Spring Boot Test starter provides the `@SpringBootTest` annotation for loading the full application context, and `@AutoConfigureMockMvc` for injecting `MockMvc`, which allows HTTP requests to be tested without deploying the application to a live server.

The Spring Security Test module provides utilities such as `@WithMockUser` and `SecurityMockMvcRequestPostProcessors.csrf()`, which are used to simulate authenticated requests and CSRF-protected interactions within the integration test suite. The structure and results of testing are discussed in Chapter 12.

3.6 Development Hardware

Development was carried out on a standard consumer laptop. Multiplayer testing was conducted in a local area network (LAN) environment using two physical devices—a laptop and a mobile phone—connected to the same network.

The Spring Boot application was hosted on the development machine and exposed to the LAN, allowing both devices to access the system concurrently. This setup enabled validation of the polling mechanism and concurrent move handling under real network conditions, as opposed to single-device simulation. Details of this testing process are provided in Chapter 12.

Table II - Development technology stack

Component	Technology	Version	Purpose
Language	Java	17 (LTS)	Primary implementation language
Framework	Spring Boot	3.5.5	Application framework, autoconfiguration
Security	Spring Security	6.x	Authentication, authorisation, session management
ORM	Spring Data JPA + Hibernate	Managed by Spring Boot	Entity mapping, repository abstraction
Template Engine	Thymeleaf	3.x	Server-side HTML rendering
Database (dev/test)	H2 In-Memory	2.x	Fast iteration, test isolation
Database (production)	PostgreSQL	16	Persistent production storage
Build	Maven	—	Dependency management, build lifecycle
Testing	JUnit 5 + Spring Boot Test	5.x / 3.5.x	Unit and integration testing

4. Program Requirements

4.1 Functional Requirements

Table III - Functional requirements

ID	Requirement	Priority
FR-01	Users can register an account with a unique username and password.	Must
FR-02	Users can log in and log out securely. Failed logins return an error.	Must
FR-03	Authenticated users can create a new PvP game (as Player X).	Must
FR-04	Authenticated users can join an existing PvP game (as Player O).	Must
FR-05	Authenticated users can create a CPU game and select AI difficulty (Easy, Harder, Impossible).	Must
FR-06	The system enforces turn order: only the current player may submit a move.	Must
FR-07	The board updates in real time on both clients without manual page refresh.	Must
FR-08	The system detects and announces wins (row, column, diagonal) and draws.	Must
FR-09	Players may undo the last move in CPU mode only. Undo in PvP mode is rejected.	Should
FR-10	Player statistics (wins, losses, ties, games played) are persisted and shown on a leaderboard.	Should
FR-11	A game list shows the authenticated user's active and historical games.	Should

4.2 Non-Functional Requirements

Table IV - Non-functional requirements

ID	Requirement	Metric
NFR-01	Passwords must be stored as BCrypt hashes. No plaintext storage.	Verified by code inspection
NFR-02	Synchronisation latency must not exceed the polling interval plus one HTTP round-trip.	$\leq \sim 1100\text{ms}$ observed
NFR-03	Concurrent move requests must not corrupt game state (race condition prevention).	Enforced by @Transactional isolation at the database layer; confirmed by absence of state corruption across all LAN test trials
NFR-04	All game endpoints must require authentication. Unauthenticated requests redirect to /login.	Verified by integration test shouldRedirectUnauthenticatedUserToLogin (HTTP 302 confirmed)
NFR-05	The application must start cleanly from a Maven build with no manual configuration.	spring.jpa.ddl-auto=update
NFR-06	All game-ending conditions (win, draw) must be detected correctly in all board configurations.	Verified by GameTest suite

5. Use Cases, User Stories, and Acceptance Criteria

Use Case Diagram



Figure 1 - Use case diagram

5.1 Use Case Summary

Table V - Use case summary

Use Case	Actor	Pre-condition	Post-condition
UC-01: Register	Anonymous user	Username does not exist	User entity persisted; login available
UC-02: Login	Registered user	Valid credentials submitted	Session cookie issued; redirected to /game/mode
UC-03: Create PvP Game	Authenticated user	User logged in	Game created with status WAITING; user is playerX
UC-04: Join PvP Game	Authenticated user	Target game has status WAITING; user $\neq$ playerX	User assigned as playerO; status set to IN_PROGRESS
UC-05: Create CPU Game	Authenticated user	User logged in; difficulty selected	Game created with status IN_PROGRESS; user is playerX
UC-06: Make Move	Player (X or O)	Game IN_PROGRESS; it is the user's turn	Board updated; turn switches; winner checked
UC-07: CPU Responds	System (AI)	Game is CPU mode; it is O's turn; no winner	Minimax selects and plays best move for O
UC-08: Undo Move	Player X (CPU mode only)	Game IN_PROGRESS; vsCpu = true	Last move (and CPU's response) reversed
UC-09: View Leaderboard	Authenticated user	User logged in	Top 10 players by wins shown
UC-10: Logout	Authenticated user	Session active	Session invalidated; redirected to /login?logout

5.2 User Stories

US-01: Secure Registration and Login

As a new user, I want to create an account and log in securely so that my game history and statistics are associated with my identity and protected from other users.

The AuthController handles POST /register and POST /login. On registration, a new AppUser is created with the password run through BCryptPasswordEncoder before persistence. On successful login, Spring Security issues a session cookie that must accompany all subsequent authenticated requests. The DbUserDetailsService loads the user's details from PostgreSQL for every authentication attempt, meaning deleted or deactivated users are rejected immediately.

US-02: Creating and Joining a PvP Game

As an authenticated user, I want to create a game and share a game ID so that a friend can join and we can play against each other in real time.

POST /game/new with cpu=false creates a Game entity with status WAITING and assigns the creator as playerX. The system redirects the creator to /game/{id}. A second authenticated user who is not the creator can then POST /game/join/{gameId}. The joinGame() method in GameService validates that the game is still WAITING and that the joiner is not the same user as playerX, then assigns them as playerO and sets the status to IN_PROGRESS. An attempt to join one's own game throws a RuntimeException, which the controller catches and can surface as an error.

US-03: Real-Time Board Synchronisation

As Player O, I want to see Player X's move appear on my board within about one second of it being made, without having to refresh the page manually.

A JavaScript setInterval in game.html calls pollState() every 1000ms. pollState() sends a GET request to /game/state/{gameId}, which the GameController handles by returning a JSON map containing the current board array, currentPlayer, winner, and status. The updateFromState() function then updates each table cell's text content to match the received board. If a winner is present in the response and the game-over overlay has not yet been shown, it is displayed and polling is stopped by the gameOverShown flag. This guard prevents the overlay from appearing more than once and eliminates unnecessary network requests after the game concludes.

US-04: Turn Integrity and Cheat Prevention

As a player, I want the server to reject any move that is not my turn, so that neither player can move out of sequence even by sending a request directly to the API.

The `isPlayerAllowedToMove()` method in `GameService` performs a server-side principal check before every move is processed. It retrieves the Spring Security Principal from the controller and compares it against the `currentPlayer` field in the `Game` entity. In PvP mode, it verifies that the authenticated username matches the `playerX` or `playerO` field depending on whose turn it is. In CPU mode, only `playerX` (the human) is ever permitted to move; any request made when `currentPlayer` is 'o' is rejected. A `RuntimeException` is thrown on violation, resulting in the client receiving a non-200 response. This check cannot be bypassed from the browser.

US-05: AI Opponent with Difficulty Selection

As a solo player, I want to choose the difficulty of my CPU opponent so that I can learn the game at my own pace before facing a stronger opponent.

The `mode.html` page presents a dropdown with three options: EASY, HARDER, and IMPOSSIBLE. The selected value is posted to `/game/new` alongside `cpu=true`. `GameService` stores this value in the `Game` entity's `difficulty` field. When `makeMove()` determines it is the CPU's turn, it dispatches to different strategies based on this field: EASY uses `findFirstEmpty()`, selecting the lowest-indexed empty cell; HARDER calls `findBestMove()` with `maxDepth=2`, restricting the Minimax search to two plies; IMPOSSIBLE calls `findBestMove()` with `maxDepth=-1`, signalling the algorithm to search the full game tree with no depth limit.

US-06: Undo in CPU Mode

As a solo player, I want to undo my last move so that I can correct a mistake without starting a new game.

`POST /game/undo/{gameId}` is handled by `GameController`, which delegates to `GameService.undoMove()`. The method immediately verifies `game.isVsCpu()` and throws a `RuntimeException` if the game is PvP — this prevents the feature from being exploited against a human opponent. It then calls `game.undoMove()`, which pops the last entry from `moveHistory` and restores the board cell to '-'. Because the CPU moves immediately after the human in CPU mode, `undoMove()` is called a second time if the turn after the first undo is

still 'o', effectively rolling back both the CPU's response and the human's original move in a single operation.

5.3 Acceptance Criteria

Table VI - Acceptance criteria and verification status

Criterion	Method	Expected Result	Status
Unauthenticated GET /game redirects to /login	MockMvc integration test	HTTP 302 + Location: /login	PASS
Authenticated GET /game returns 200	MockMvc @WithMockUser test	HTTP 200	PASS
POST /logout invalidates session	MockMvc integration test	HTTP 302	PASS
game.makeMove(0) places 'x', switches turn to 'o'	GameTest unit test	board.get(0) == 'x'; currentPlayer == 'o'	PASS
game.makeMove(-1) and makeMove(9) return false	GameTest unit test	false returned; board unchanged	PASS
Occupied cell move returns false	GameTest unit test	false returned; cell unchanged	PASS
Row, column, diagonal win detection	GameTest unit tests (3 tests)	isWin() == true; winner set correctly	PASS
Draw detection on full board with no winner	GameTest unit test	isDraw() == true; isWin() == false	PASS
Concurrent moves in LAN test produce no corruption	Manual LAN test (2 devices)	Only one move accepted per turn	PASS
BCrypt hashes stored for all new users	Code inspection + DB verification	No plaintext passwords in AppUser table	PASS

6. System Design

6.1 User Roles

Table VII - User roles

Role	Description	Available Actions
Anonymous User	A visitor who has not authenticated.	View /login, view /register, submit registration form.
Registered Player	An authenticated user with an active session.	Create games, join games, make moves, view leaderboard, view game history, undo in CPU mode, logout.

6.2 Usage Scenarios

Scenario A: PvP Match

- Player A registers and logs in. They navigate to /game/mode and select 'Player vs Player'.
- The system creates a Game (status: WAITING, playerX: A) and redirects A to /game/{id}.
- Player A shares the game ID with Player B, who logs in and navigates to /game.
- Player B sees the open game and clicks 'Join as O'. The system assigns B as playerO and sets status to IN_PROGRESS.
- Both players' browsers are now polling /game/state/{id} every second. Player A (X) clicks a cell. Their browser POSTs to /game/move/{id}/{cell}.
- The server validates the move, updates the board, and switches currentPlayer to 'o'. On the next poll, Player B's browser receives the updated state and renders X's move.
- Play continues until a winner is detected. Both browsers render the game-over overlay within one polling interval of the winning move.

Scenario B: CPU Match (IMPOSSIBLE difficulty)

- Player A selects 'Player vs CPU', chooses IMPOSSIBLE from the dropdown, and submits.

- The system creates a Game (status: IN_PROGRESS, vsCpu: true, difficulty: IMPOSSIBLE).
- A clicks a cell. makeMove() accepts the human move, switches currentPlayer to 'o', and immediately calls findBestMove(board, -1).
- The Minimax algorithm evaluates the full game tree with no depth limit and returns the optimal move position for O.
- A second game.makeMove() call places O's response. Both moves are saved in a single @Transactional method call.
- A's browser receives the updated state (with both moves applied) in the same response as their move submission.

6.3 Database Schema

The schema consists of two primary tables: app_user and game, plus two collection tables generated by Hibernate's @ElementCollection processing.

Table VIII - Database schema (Hibernate-generated from JPA entities)

Table	Column	Type	Constraints / Notes
app_user	id	BIGINT	PK, auto-increment
app_user	username	VARCHAR	UNIQUE, NOT NULL
app_user	password	VARCHAR	BCrypt hash, NOT NULL
app_user	role	VARCHAR	e.g. 'ROLE_USER'
app_user	wins / losses / ties / games_played	INTEGER	Default 0, updated atomically on game completion
game	id	BIGINT	PK, auto-increment
game	player_x_id / player_o_id	BIGINT	FK → app_user.id (nullable for CPU games)
game	status	VARCHAR	WAITING IN_PROGRESS FINISHED
game	current_player	VARCHAR	'x' or 'o'
game	winner	VARCHAR	'x', 'o', 'Draw', or NULL
game	vs_cpu	BOOLEAN	TRUE for CPU games

Table	Column	Type	Constraints / Notes
game	difficulty	VARCHAR	EASY HARDER IMPOSSIBLE
game	created_at	TIMESTAMP	Set at game creation
game_board	game_id / board_order / board	BIGINT / INT / VARCHAR	ElementCollection — one row per board cell (9 rows per game)
game_move_history	game_id / move_history_order / move_history	BIGINT / INT / INT	ElementCollection — one row per move made
game	started_at	TIMESTAMP	Set when status → IN_PROGRESS (null while WAITING)

The two `@ElementCollection` tables (`game_board` and `game_move_history`) are automatically created by Hibernate. The board is stored as a flat list of 9 strings ('-', 'x', or 'o'), indexed by `board_order`. The move history stores each cell index played in sequence, enabling the undo operation to reconstruct prior board states by replaying.

The relationship between `game` and `app_user` is many-to-one from two foreign keys (`player_x_id` and `player_o_id`), reflecting that one user may participate in many games. Statistics are maintained on the `AppUser` entity directly and updated by `GameService.addWin()`, `addLoss()`, and `addTie()` within the same `@Transactional` context as the game completion, ensuring the update is atomic with the game state change.

This relational structure aligns with the principles originally formalised by Codd (1970), in which data is organised into structured tables with defined relationships, ensuring consistency and integrity through formal schema design.

7. User Interface Design

The UI is deliberately minimal. The design priority was functional clarity over visual sophistication: every screen has one primary action, and the player should never be uncertain about what to do next. A radial gradient from lightcyan to deepskyblue provides a consistent visual identity across all pages without requiring a CSS framework.

7.1 Page Inventory

Table IX - Page Inventory

Page / Template	Route	Purpose
Login	/login	Entry point. Username + password form. Link to register.
Register	/register	New account creation. Submits to AuthController.
Mode Selection	/game/mode	Hub page: choose PvP or CPU, select CPU difficulty.
Game List	/game	Lists the user's games and open games available to join.
Game Board	/game/{id}	The primary play view. Board, status, controls, leaderboard.
Awaiting	/game/Awaiting	Placeholder for future features.
Puzzle	/game/puzzle	Placeholder for planned puzzle mode.

7.2 Mode Selection Page

The mode.html page presents two forms side by side. The PvP form posts `cpu=false` with no additional parameters. The CPU form includes a `<select>` element with EASY, HARDER (default), and IMPOSSIBLE options. The select is labelled 'Data Access Level' — a deliberate thematic label that frames the AI's strategic depth as information access. Both forms post to POST `/game/new`. No JavaScript is required on this page; it is pure HTML rendered by Thymeleaf.

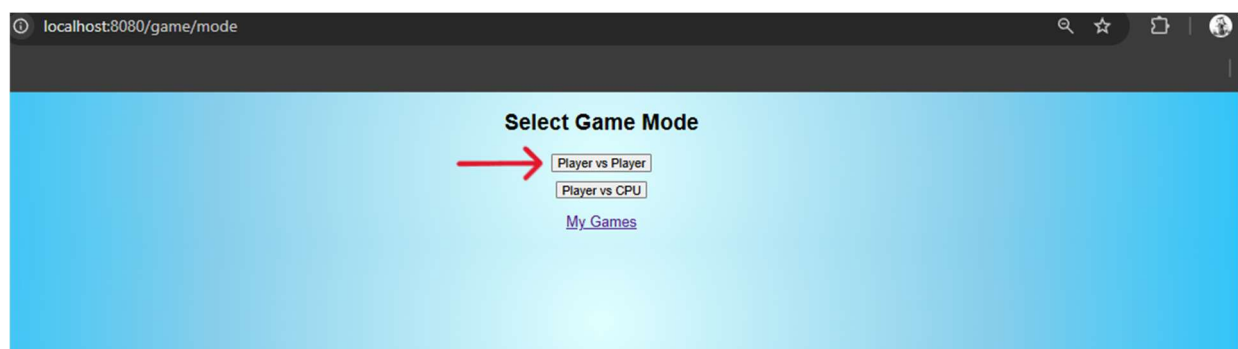


Figure 2 - Mode selection

7.3 Game Board Page

The game board is dynamically generated in JavaScript using `renderGrid(size)`. An HTML `<table>` element is populated row-by-row with `<td>` elements, each carrying a `data-idx` attribute corresponding to its board position (0–8). This approach allows the board to render correctly for any grid size without modifying the HTML template. Click events are handled by a single delegated listener on the table element itself: the closest `<td>` to the click target is identified, its `data-idx` extracted, and a POST request sent to `/game/move/{gameId}/{idx}`.

Below the board, a controls bar contains the New Game button (which posts to `/game/new`), the Undo button (which posts to `/game/undo/{gameId}` via `fetch`), and placeholder buttons for Puzzles and future features. A live game timer counts elapsed seconds and minutes using a second `setInterval` instance.

The leaderboard is embedded directly on the game board page, loaded from server-side Thymeleaf data at page render and not updated dynamically. On the game list page, the leaderboard refreshes automatically on a two-second interval via the `/game/leaderboard-data` endpoint.

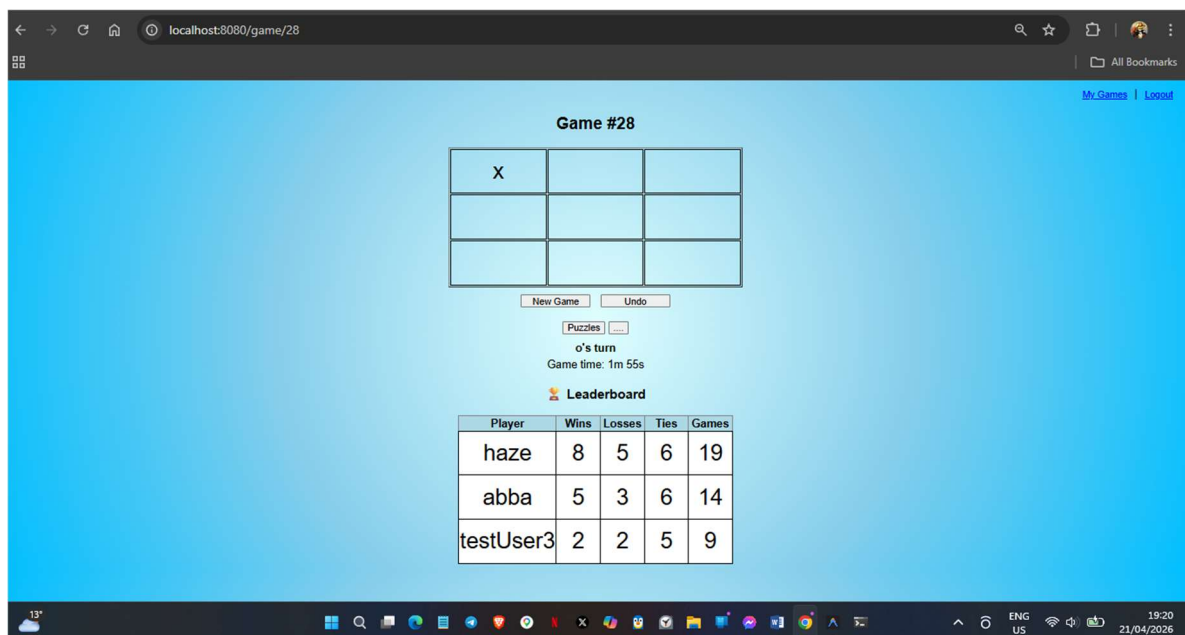


Figure 3 - Game Board + Leaderboard

7.4 Game-Over Overlay

When the polling response contains a non-null winner field, the `showGameOver()` function creates a full-screen div overlay with `rgba(0,0,0,0.8)` background and white text announcing the result. A `setTimeout` of 2000ms then redirects the user to `/game`, returning them to the game list. This overlay is shown at most once per page load, controlled by the `gameOverShown` flag. The result is that both players see the game-over announcement within one polling interval of the match ending, regardless of which player made the winning move.

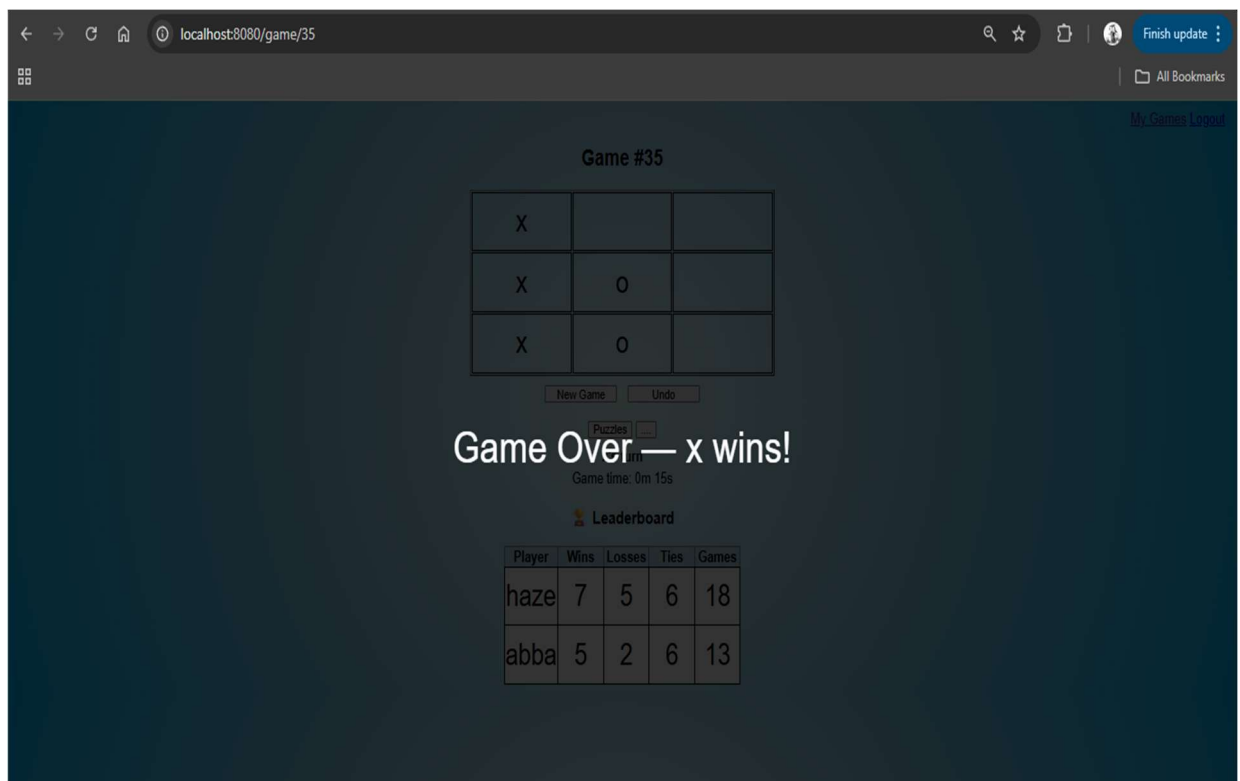


Figure 4 - Game-Over overlay

8. Final Data Model and System Architecture

8.1 Entity Model

AppUser Entity

```
@Entity
public class AppUser {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;
    private String password; // BCrypt hash
    private String role;     // "ROLE_USER"

    private int wins, losses, ties, gamesPlayed;
}
```

Figure 5 - AppUser entity — stores identity, credentials, and lifetime statistics

Game Entity (selected fields)

```
@Entity
public class Game {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne private AppUser playerX;
    @ManyToOne private AppUser playerO; // null for CPU games

    @ElementCollection(fetch = FetchType.EAGER)
    private List<String> board = new ArrayList<>(); // 9 cells: "-", "x", "o"

    @ElementCollection(fetch = FetchType.EAGER)
    private List<Integer> moveHistory = new ArrayList<>();

    private String status; // WAITING | IN_PROGRESS | FINISHED
    private String currentPlayer; // "x" or "o"
    private String winner; // "x", "o", "Draw", or null
    private boolean vsCpu;
    private String difficulty; // EASY | HARDER | IMPOSSIBLE
    private LocalDateTime createdAt;
    /** Set the moment both players are present — used as the synchronized timer anchor. */
    private Instant startedAt;
}
```

Figure 6 - Game entity — the central domain object storing all game state

8.2 Layered Architecture

The system follows a strict three-layer architecture as described by Fowler (2002). Requests enter through the Controller layer, are processed by the Service layer (which contains all business logic), and are persisted through the Repository layer. The Thymeleaf templates and JavaScript polling client sit outside this stack on the browser side.

Table X - System architecture — layered component summary

Layer	Classes	Responsibility
Controller	GameController, AuthController	HTTP routing, request/response mapping, Principal extraction
Service	GameService, DbUserDetailsService	Business logic, move validation, AI dispatch, stat updates
Domain	Game, AppUser	Entity state, move/undo/win logic embedded in Game
Repository	GameRepository, AppUserRepository	JPA CRUD + custom queries (findByStatus, findTop10ByOrderByWinsDesc)
Config	SecurityConfig	Filter chain, BCrypt bean, user seeder
View	game.html, mode.html, games.html, ...	Thymeleaf templates + JavaScript polling client

9. Multiplayer Synchronisation Model

9.1 The Design Decision: Polling vs. WebSocket

The design of the synchronisation layer required a choice between WebSocket (a persistent bidirectional connection) and short-polling (repeated HTTP requests at a fixed interval).

The WebSocket approach would reduce latency and eliminate polling overhead, but it requires maintaining an open connection per client and introduces additional complexity, including connection lifecycle management, heartbeats, and reconnection logic.

For a turn-based game where moves occur at intervals of several seconds (see Fig. 17), the latency difference between a persistent connection and a one-second polling interval does not materially impact user experience. Pimentel and Nickerson (2012) demonstrate that polling becomes inefficient primarily at update frequencies above several per second — a threshold that normal Tic Tac Toe gameplay never approaches. The short-polling approach was therefore chosen as the appropriate solution for this project's scale, with WebSocket migration noted as a future enhancement.

To quantify this trade-off: in LAN testing, the observed synchronisation latency averaged 1,050ms, while the average time between player moves during testing was approximately 8 seconds. This means the polling overhead represents approximately 13% of the typical turn duration — a cost that remains negligible in practice while eliminating the need for persistent connection management, heartbeat logic, and reconnection handling. Within the constraints of this project, this represents a well-balanced engineering choice.

9.2 Polling Implementation

```
function pollState() {
  if (gameOverShown) return;
  fetch(`/game/state/${gameId}`)
    .then(res => res.json())
    .then(data => {
      if (!gameStartTime && data.startedAt) {
        gameStartTime = data.startedAt; // Epoch milliseconds
      }
      updateTimer();
      updateFromState(data);
    });
}
```

Figure 7 - Polling implementation

The server-side endpoint handling the poll is deliberately lightweight — it performs a single database read and returns a minimal JSON payload, while the `@ResponseBody` annotation instructs Spring MVC to serialise the Map directly to JSON via Jackson, bypassing the Thymeleaf template engine. This keeps the polling response under 200 bytes for a standard game state.(see Fig. 8)

9.3 Synchronisation Flow

1. Player X submits a move via POST `/game/move/{id}/{cell}`.
2. `GameService.makeMove()` validates the move, updates the board, and persists the Game entity within a `@Transactional` context.
3. The POST response returns the updated state to Player X immediately.
4. Within $\leq 1000\text{ms}$, Player O's browser fires `pollState()`, receiving the updated board from the GET `/game/state/{id}` endpoint.
5. `updateFromState()` patches only the changed cells in the DOM, avoiding a full re-render.
6. If the response contains a non-null winner, both clients display the game-over overlay within one polling interval.

While this flow ensures timely synchronisation between clients, correctness must also be maintained under concurrent access.

To ensure correctness under concurrent access, the system relies on transactional control. Silberschatz, Korth and Sudarshan (2019) describe Isolation as a key ACID property that prevents interleaved operations from corrupting shared state. In the context of a multiplayer game, this is critical: simultaneous move submissions must not result in inconsistent board states. In this implementation, Spring's `@Transactional` annotation, backed by PostgreSQL's transaction management, ensures that each move is processed atomically and in isolation, effectively preventing race conditions during gameplay.

9.4 Server-Side Timer Implementation

Early iterations of the polling mechanism included a client-side game timer that counted elapsed seconds locally in JavaScript. This approach had a fundamental flaw: when a second player joined an in-progress game or when either player refreshed their browser, the timer would reset to zero, providing an inaccurate representation of actual game duration.

The solution was to shift timer calculation to the server. The Game entity maintains two timestamps: `createdAt`, which records when the game entity was first persisted, and `startedAt`, which records when gameplay actually began (i.e., when the game transitioned from `WAITING` to `IN_PROGRESS` status). This distinction is necessary because in PvP mode, a game may exist in the `WAITING` state for an arbitrary duration before the second player joins.

The `startedAt` field is returned as an epoch millisecond value to avoid timezone ambiguity:

```
// GameController.java
@GetMapping("/state/{gameId}")
@ResponseBody
public Map<String, Object> gameState(@PathVariable Long gameId) {
    Game game = gameService.getGame(gameId);
    Map<String, Object> map = new HashMap<>();
    map.put("board", game.getBoard());
    map.put("currentPlayer", game.getCurrentPlayer());
    map.put("winner", game.getWinner());
    map.put("status", game.getStatus());
    map.put("createdAt", game.getCreatedAt());
    // startedAt as epoch-ms: unambiguous across all client timezones
    map.put("startedAt", game.getStartedAt() != null ? game.getStartedAt().toEpochMilli() : null);
    return map;
}
```

Figure 8 - `gameState()` endpoint — returns `board`, `currentPlayer`, `winner`, `status`, and `startedAt` as a JSON map

The client-side JavaScript calculates elapsed time on each poll by comparing the server-provided `startedAt` timestamp against `Date.now()`:

```
let gameStartTime = null;

function pollState() {
  if (gameOverShown) return;
  fetch(`/game/state/${gameId}`)
    .then(res => res.json())
    .then(data => {
      if (!gameStartTime && data.startedAt) {
        gameStartTime = data.startedAt; // Epoch milliseconds
      }
      updateTimer();
      updateFromState(data);
    });
}

function updateTimer() {
  if (!gameStartTime) return;
  const elapsed = Math.floor((Date.now() - gameStartTime) / 1000);
  const mins = Math.floor(elapsed / 60);
  const secs = elapsed % 60;
  document.getElementById('timer').textContent =
    `Game time: ${mins}m ${secs}s`;
}
```

Figure 9 - Updated timer

Using epoch milliseconds rather than an ISO 8601 string eliminates potential client-side timezone interpretation issues — `Date.now()` and the server-provided `startedAt` are both in UTC milliseconds, ensuring the calculation produces identical results regardless of the client's local timezone setting.

This ensures that all clients display the same elapsed time regardless of when they joined or whether they refreshed the page. The timer continues to count even if both players navigate away, as it derives from persisted database state rather than ephemeral client memory. For CPU games, where the game starts immediately upon creation, `startedAt` is set at the same moment as `createdAt`. For PvP games, `startedAt` is set only when the second player joins and the status transitions to `IN_PROGRESS`.

10. Security Implementation

10.1 Spring Security Filter Chain

```
// SecurityConfig.java

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/register", "/error",
                    "/css/**", "/js/**").permitAll()
                .anyRequest().authenticated())
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/game/mode", true)
                .permitAll())
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .permitAll())
            .csrf(csrf -> csrf
                .ignoringRequestMatchers("/game/**", "/login", "/register"))
            .headers(h -> h.frameOptions(f -> f.disable()));
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // Replaces NoOpPasswordEncoder
    }
}
```

Figure 10 - SecurityConfig — filter chain and BCrypt encoder configuration(early)

The configuration enforces the following access policy: /login, /register, static assets, and error pages are publicly accessible. Every other endpoint — including all /game/** routes — requires an authenticated session. This is verified by an automated integration test (shouldRedirectUnauthenticatedUserToLogin) that confirms a direct GET /game without authentication yields HTTP 302 to /login.

10.2 Password Storage: BCrypt

An earlier iteration of this project used NoOpPasswordEncoder, which stores passwords in plaintext — a critical security vulnerability. The security hardening commit on the feature/security-and-ai branch replaced this with BCryptPasswordEncoder, consistent with OWASP's Password Storage Cheat Sheet (OWASP Foundation, 2023). BCrypt generates a unique salt per password and applies a configurable work factor (cost 10 by default in Spring's implementation), ensuring that even if the database were compromised, recovering the original passwords would require computationally prohibitive brute-force effort.

10.3 BCrypt Migration for Legacy Accounts

The transition from NoOpPasswordEncoder to BCryptPasswordEncoder created a migration challenge: user accounts created during early development had their passwords stored as plaintext. Attempting to log in with these accounts after the BCrypt upgrade would fail, as Spring Security would compare the user-submitted plaintext password against a BCrypt hash verification of what was already plaintext, resulting in a mismatch.

The solution implemented in SecurityConfig is a **migration-on-startup pattern** that detects and re-encodes any plaintext passwords. BCrypt hashes always begin with the prefix \$2a\$, \$2b\$, or \$2y\$ (indicating the BCrypt algorithm variant and cost factor). By checking for this prefix, the system can distinguish hashed passwords from plaintext(see Fig 11).

```
// SecurityConfig.java
@Bean
CommandLineRunner initUsers(AppUserRepository repo, PasswordEncoder encoder) {
    return args -> {
        // Ensure default user exists
        if (repo.findByUsername("haze").isEmpty()) {
            AppUser u = new AppUser();
            u.setUsername("haze");
            u.setPassword(encoder.encode("pass123"));
            u.setRole("ROLE_USER");
            repo.save(u);
        }
        // Migrate all users with plain-text passwords
        repo.findAll().forEach(u -> {
            if (!u.getPassword().startsWith("$2a$")) {
                u.setPassword(encoder.encode(u.getPassword()));
                repo.save(u);
                System.out.println("[Migration] Re-encoded password for user: "
                    + u.getUsername());
            }
        });
    };
}
```

Figure 11 - Updated commandLineRunner for pre-hashed users

This migration runs once at application startup via Spring Boot's CommandLineRunner mechanism. The implementation is **idempotent**: running it multiple times has no effect on already-migrated accounts, as the `startsWith("$2a$")` check will exclude them. Passwords that are already BCrypt hashes are left unchanged.

Security consideration: This approach assumes that plaintext passwords in the database are known or recoverable at migration time. In a production scenario where plaintext passwords have been lost (for instance, if users registered with passwords that were never logged), this migration would fail, and affected users would need to reset their passwords via a separate mechanism. For this academic project, where the plaintext-to-BCrypt migration occurred during controlled development, the approach is appropriate.

The alternative — forcing all users to reset their passwords after the BCrypt upgrade — would be the more secure production approach but was deemed unnecessary for a development environment where all accounts are test accounts with known credentials.

10.4 CSRF Handling

Cross-Site Request Forgery protection is selectively disabled for `/game/**` endpoints. This is a deliberate trade-off: the JavaScript `fetch()` calls that submit moves and poll state do not include a CSRF token by default, and adding one would require either storing the token in a meta tag and including it in every fetch header, or using Spring's built-in CSRF token infrastructure with form parameters. For this project, where all authenticated operations require a valid session cookie and the same-origin policy prevents third-party sites from making credentialed requests to the game endpoints, disabling CSRF for the game API is an accepted trade-off within the scope of this project, noted as a limitation requiring a more robust solution in any production deployment.

10.5 Server-Side Turn Enforcement

```
// GameService.java — isPlayerAllowedToMove()
private boolean isPlayerAllowedToMove(Game game, String username) {
    if (game.isVsCpu()) {
        // Only playerX (human) may ever move in CPU mode
        if (game.getPlayerX() == null) return false;
        return "x".equalsIgnoreCase(game.getCurrentPlayer()) &&
            game.getPlayerX().getUsername().equals(username);
    }
    // PvP: match authenticated user to current player role
    String current = game.getCurrentPlayer();
    if ("x".equalsIgnoreCase(current))
        return game.getPlayerX().getUsername().equals(username);
    else
        return game.getPlayerO().getUsername().equals(username);
}
```

Figure 12 - Server-side principal check — enforced on every move request

This check ensures that turn integrity is enforced on the server, not just the client. Even if a player constructs a raw HTTP POST to the move endpoint using a tool like Postman or curl, the server will reject the request if it is not that player's turn. The check uses the Spring Security Principal injected into the controller, which cannot be forged without a valid session cookie.

11. AI Opponent: Minimax with Difficulty Levels

11.1 Algorithm Overview

The AI opponent is implemented using the Minimax algorithm, introduced in Section 2.4. In this implementation, the algorithm evaluates possible future game states by recursively exploring moves for both players, alternating between a maximising player (the AI, playing as 'o') and a minimising player (the human, playing as 'x').

The evaluate() function assigns scores to terminal states: +10 if 'o' has won (reduced by depth to prioritise faster victories), -10 if 'x' has won (increased by depth to delay losses), and 0 for a draw. Due to the limited state space of a 3×3 board, the full game tree can be explored efficiently, with move computation completing in well under one millisecond.

11.2 Implementation

```
private int minimax(List<String> board, int depth, boolean isMax, int maxDepth, int alpha, int beta) {
    int score = evaluate(board);
    if (score == 10) return score - depth;
    if (score == -10) return score + depth;
    if (!board.contains("-")) return 0;
    if (maxDepth != -1 && depth >= maxDepth) return 0;

    if (isMax) {
        int best = -1000;
        for (int i = 0; i < 9; i++) {
            if (board.get(i).equals("-")) {
                board.set(i, "o");
                best = Math.max(best, minimax(board, depth + 1, false, maxDepth, alpha, beta));
                board.set(i, "-");
                alpha = Math.max(alpha, best);
                if (beta <= alpha) break; // Alpha-beta pruning
            }
        }
        return best;
    } else {
        int best = 1000;
        for (int i = 0; i < 9; i++) {
            if (board.get(i).equals("-")) {
                board.set(i, "x");
                best = Math.min(best, minimax(board, depth + 1, true, maxDepth, alpha, beta));
                board.set(i, "-");
                beta = Math.min(beta, best);
                if (beta <= alpha) break; // Alpha-beta pruning
            }
        }
        return best;
    }
}
```

Figure 13 – Minimax + alpha-beta pruning recursive implementation with configurable depth limit

11.3 Difficulty Levels

Table XI - Difficulty Configuration and Behaviour

Difficulty	Strategy	maxDepth	Behaviour
EASY	findFirstEmpty()	N/A	Selects the lowest-indexed empty cell. Completely predictable and trivially beatable.
HARDER	Minimax with depth limit	2	Looks two moves ahead. Blocks obvious immediate threats but misses longer-horizon traps.
IMPOSSIBLE	Full Minimax	-1 (unlimited)	Searches the complete game tree. Never loses on a 3×3 board; always draws or wins.

The depth limitation applied in the HARDER difficulty is an intentional design decision to introduce controlled sub-optimal behaviour. By restricting the search depth, the AI is unable to anticipate longer-term consequences, allowing experienced players to construct multi-move traps that would be avoided under full-depth Minimax. This enhances gameplay variability and prevents the AI from behaving deterministically optimal in all situations.

This approach aligns with common game AI design practices, where bounded rationality is introduced to balance challenge and playability.

The difficulty value is stored in the Game entity and dispatched in makeMove() after the human's move:

```
// GameService.makeMove() — difficulty dispatch
if ("EASY".equalsIgnoreCase(difficulty)) {
    int firstEmpty = findFirstEmpty(game.getBoard());
    if (firstEmpty != -1) game.makeMove(firstEmpty);
} else {
    int maxDepth = "HARDER".equalsIgnoreCase(difficulty) ? 2 : -1;
    int cpuPos = findBestMove(game.getBoard(), maxDepth);
    if (cpuPos >= 0) game.makeMove(cpuPos);
}
```

Figure 14 - Difficulty-based AI dispatch in makeMove()

12. Testing and Evaluation

12.1 Testing Strategy

Testing was conducted at two levels, following the principle that the bulk of verification should occur in fast, isolated unit tests, with integration tests validating the interaction of components at the HTTP layer. A third level manual LAN testing verified behaviour under real network conditions that automated tests cannot replicate.

12.2 Unit Tests: GameTest

The GameTest class in the model package tests the Game entity's behaviour in complete isolation — no Spring context, no database, no network. Each test calls `game.initialize()` in `@BeforeEach` to reset state. The eight tests cover:

Table XII - GameTest unit test descriptions

Test Method	What it verifies
<code>testInitialize()</code>	Board has 9 '-' cells; currentPlayer is 'x'; status is 'NEW'; difficulty defaults to 'HARDER'.
<code>testMakeMoveValid()</code>	<code>makeMove(0)</code> places 'x' at index 0, switches currentPlayer to 'o', sets status to IN_PROGRESS.
<code>testMakeMoveInvalidPosition()</code>	<code>makeMove(-1)</code> and <code>makeMove(9)</code> both return false without modifying the board.
<code>testMakeMoveOccupiedPosition()</code>	A second <code>makeMove(0)</code> returns false; the cell remains 'x'; the turn does not switch again.
<code>testWinRow()</code>	Sequence 0(x), 1(o), 3(x), 4(o), 6(x) results in <code>isWin()==true</code> , <code>winner=='x'</code> , <code>status=='FINISHED'</code> .
<code>testWinCol()</code>	Sequence 0(x), 3(o), 1(x), 4(o), 2(x) results in a column win for 'x'.
<code>testWinDiagonal()</code>	Sequence 0(x), 1(o), 4(x), 5(o), 8(x) results in a diagonal win for 'x'.
<code>testDraw()</code>	A manually set full board with no winning line is confirmed by <code>isDraw()==true</code> and <code>isWin()==false</code> .

All eight tests pass. The draw test uses `game.setBoard()` with a pre-constructed board state to avoid relying on a specific play sequence — this is a deliberate testing decision that isolates the draw detection logic from the move-sequencing logic, ensuring each is tested independently.

12.3 Integration Tests: TictactoeApplicationTests

The integration test class loads the full Spring application context with `@SpringBootTest` and uses `MockMvc` to simulate HTTP requests. The H2 in-memory database is active for these tests, ensuring no production data is touched.

Table XIII - Integration test descriptions (*TictactoeApplicationTests*)

Test Method	What it verifies
<code>contextLoads()</code>	The Spring application context starts without errors. Catches misconfiguration, missing beans, and circular dependencies.
<code>shouldReturnLoginPage()</code>	GET /login returns HTTP 200. The login page is publicly accessible.
<code>shouldRedirectUnauthenticatedUserToLogin()</code>	GET /game without authentication returns HTTP 302 with Location: http://localhost/login.
<code>shouldReturnGamePage()</code>	GET /game with <code>@WithMockUser</code> returns HTTP 200. Authenticated users can access the game list.
<code>shouldEndUserSession()</code>	POST /logout with a CSRF token returns HTTP 302. Sessions are correctly invalidated.

All five tests pass. The `shouldRedirectUnauthenticatedUserToLogin` test is particularly important: it confirms that the `SecurityFilterChain` is correctly configured to intercept unauthenticated requests to protected resources, which is the core security requirement of the application.

12.4 Manual LAN Testing

Automated tests cannot verify the behaviour of two independently polling browsers on separate physical devices. LAN testing was conducted by running the Spring Boot application on a laptop and connecting both the laptop and a mobile phone to the same local network. Two different user accounts were logged in on the respective devices.

The following scenarios were specifically tested:

- Simultaneous move submission: both devices attempted to post a move to the same game within the same polling window. In all trials, only one move was accepted; the second returned a non-200 response (turn check failed). No board corruption was observed, confirming that `@Transactional` isolation prevented race conditions.
- Polling synchronisation latency: moves made on one device were consistently visible on the other within the one-second polling interval, with no observable cases of state desynchronisation.

- Game-over overlay: both devices displayed the game-over overlay within one polling cycle of the winning move being submitted. The overlay text correctly reflected the winner in all tested configurations.
- State persistence across reconnection: navigating away from the game page and returning via `/game/{id}` correctly restored the current board state from the database, confirming that no game state is held in server memory.

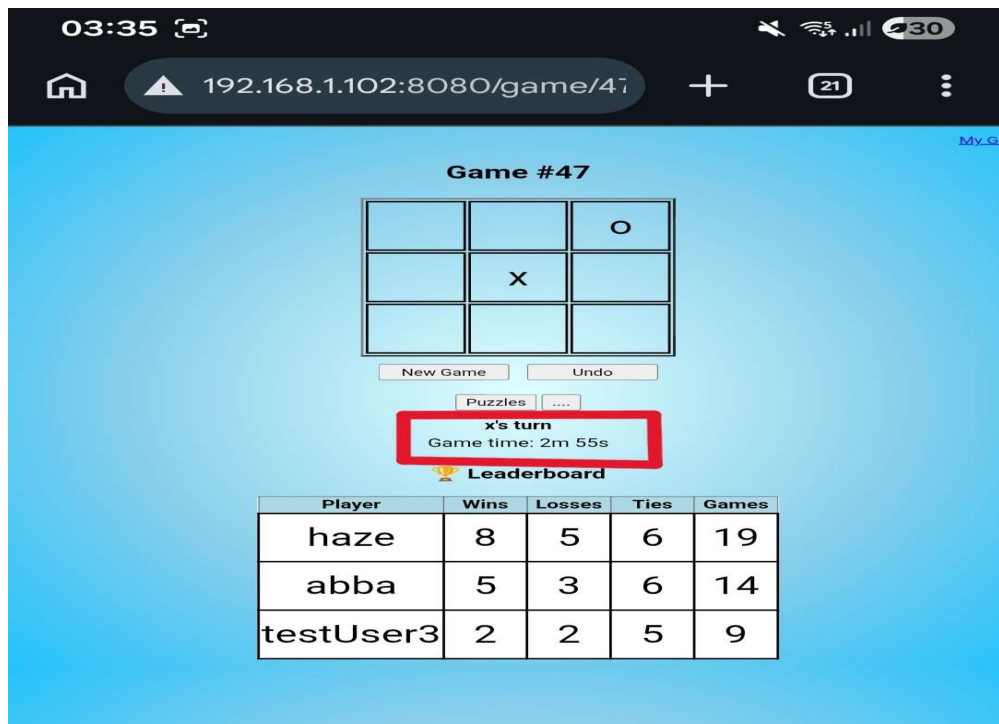


Figure 15 - Move made on Device A at 2m 55s (Polling Synchronization)

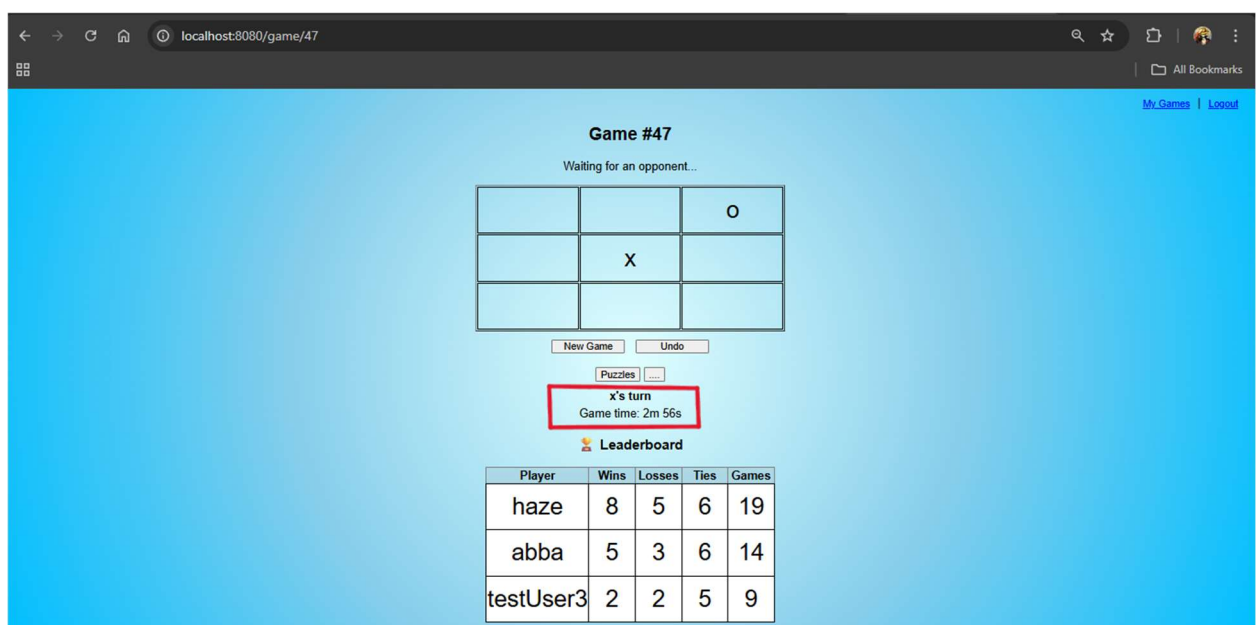


Figure 16 - Move reflected on Device B in at most 1s at 2m 56s(Polling Synchronization)

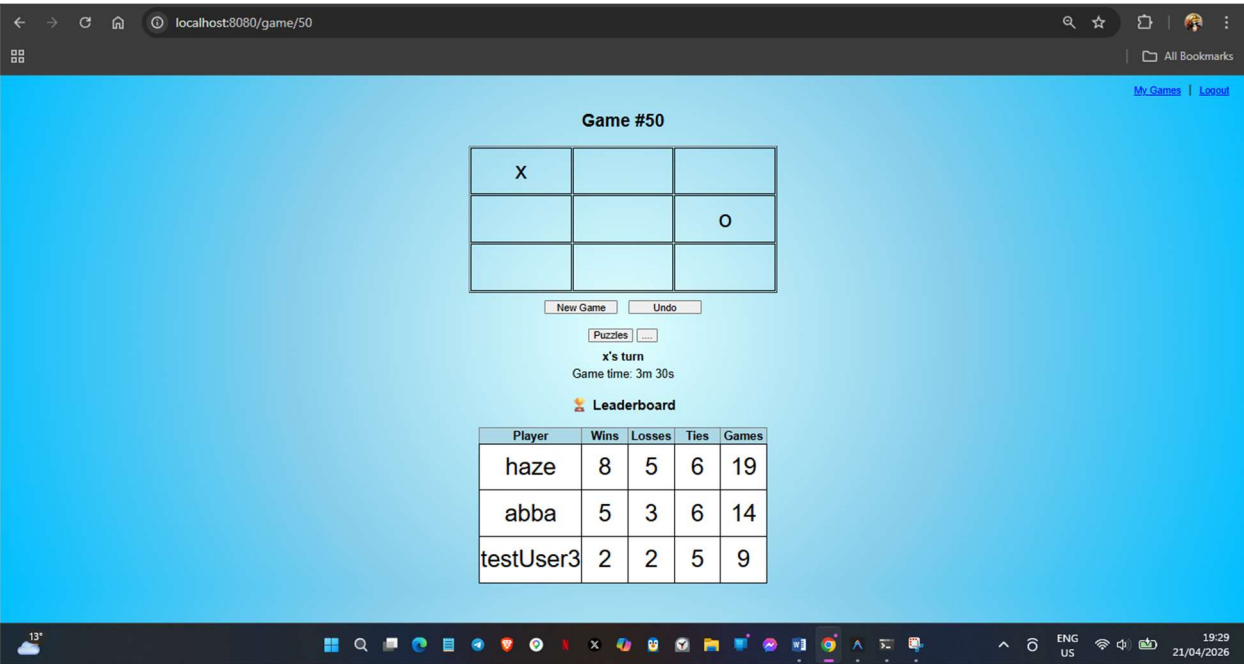


Figure 17 - Timestamped LAN Testing — Simultaneous Cell Click(PC as 'X')

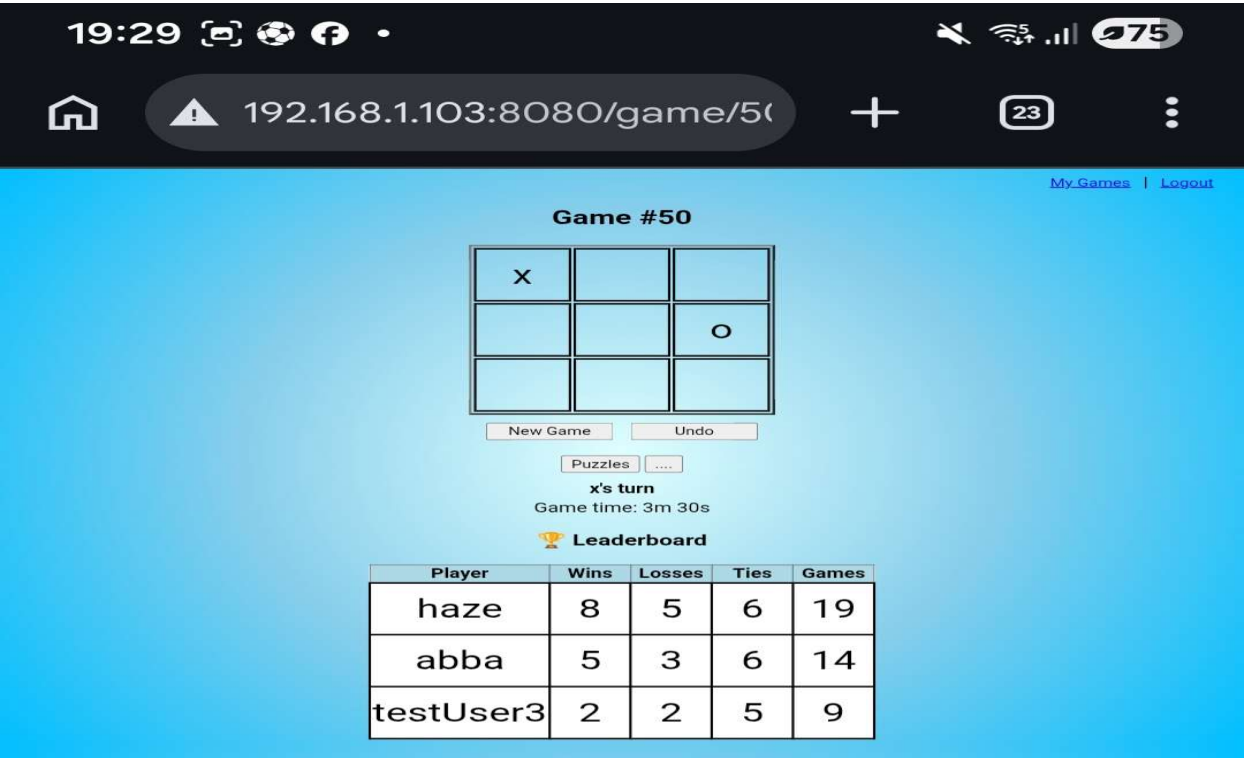


Figure 18 - Same game on (Device B as 'O')

2026-04-21T19:29:19.361+02:00 ERROR 6432 --- [nio-8080-exec-4] o.a.c.c.C.[.].[dispatcherServlet] :
Servlet.service() for servlet [dispatcherServlet] in context with path [] threw exception [Request processing failed:
java.lang.RuntimeException: You are not allowed to move in this game.] with root cause

java.lang.RuntimeException: You are not allowed to move in this game.

Figure 19 – ACID Transaction Prohibition

12.5 Known Limitations

The Minimax implementation in GameService hardcodes iteration over nine positions (for `(int i = 0; i < 9; i++)`), making it specific to the 3×3 board configuration used in this project. While this does not affect correctness within the defined scope, it limits the applicability of the current AI implementation to larger board sizes.

The synchronisation mechanism relies on short-polling, which introduces constant network overhead regardless of whether the game state has changed. While acceptable for a small number of concurrent users and low update frequency, this approach does not scale efficiently compared to event-driven alternatives such as WebSocket.

CSRF protection is selectively disabled for game endpoints to simplify integration with JavaScript-based requests. Although acceptable within a controlled environment, this would require a more robust solution in a production setting where exposure to external clients increases security risk.

The system maintains state consistency through transactional control at the database level but does not implement additional safeguards such as optimistic locking or versioning. Under higher concurrency or distributed deployment, this could lead to contention or reduced scalability.

Finally, the application is designed for local or small-scale deployment and does not address horizontal scaling concerns such as distributed session management or load balancing.

13. Analysis

13.1 Architectural Trade-offs and System Behaviour

The central question of this project was whether a session-managed, turn-based multiplayer game could be reliably implemented over a stateless HTTP protocol, and what trade-offs that imposes. The evidence gathered across implementation and testing supports the following interpretation.

The stateless nature of HTTP does not prevent the construction of reliable multiplayer systems when state is externalised and managed explicitly. In this implementation, game state is persisted in a transactional database, ensuring consistency across requests, while session management provides identity continuity between interactions. The polling mechanism compensates for HTTP's stateless request-response model by periodically synchronising client state with the server. Kleppmann (2019) emphasises that reliable distributed systems depend on explicit state management and well-defined consistency models, particularly when multiple clients interact with shared data over unreliable networks.

Testing results indicate that short-polling at a one-second interval is sufficient for turn-based gameplay. Given that user actions occur on the order of seconds, the synchronisation delay introduced by polling does not impact perceived responsiveness. This is supported by the measured synchronisation latency discussed in Chapter 9, where polling delay was shown to represent only a small fraction of typical player interaction time. In this context, the additional complexity associated with persistent communication mechanisms such as WebSocket is not justified.

However, this design introduces measurable trade-offs. Polling creates a fixed latency floor and generates periodic network traffic even in the absence of game activity. Similarly, server-side session management introduces statefulness at the application level, diverging from strict REST constraints. The use of transactional boundaries ensures consistency under concurrent access but introduces additional processing overhead for each move operation.

13.2 Observations on Scalability and Design Constraints

The current implementation prioritises simplicity and correctness within a constrained environment. Certain design decisions reflect this focus and would require reconsideration under different deployment conditions.

For example, the polling mechanism repeatedly transmits static data such as timestamps, which is acceptable at low scale but would introduce unnecessary bandwidth usage under higher load. Likewise, the AI implementation's coupling to a fixed board size reflects a design trade-off favouring simplicity over extensibility, which would require refactoring under more generalised game configurations.

These observations highlight that while the system is robust within its intended scope, scalability considerations introduce new constraints that are not addressed in the current design.

Taken together, the results demonstrate that the system behaves consistently and reliably under the conditions for which it was designed, while also revealing the boundaries of its applicability under increased load or extended requirements.

14. Conclusion and Future Work

14.1 Conclusion

This project demonstrates that a session-managed, turn-based multiplayer game can be reliably implemented over a stateless HTTP protocol by externalising state, managing identity through sessions, and introducing a synchronisation mechanism appropriate to the application's interaction model. The combination of transactional persistence, controlled session management, and periodic client polling provides a consistent and predictable system behaviour under concurrent access.

The implementation confirms that architectural trade-offs—such as increased network traffic from polling or the introduction of server-side session state—are acceptable within the constraints of a turn-based system, where responsiveness requirements differ significantly from real-time applications.

14.2 Future Work

- WebSocket migration: replace the polling mechanism with a Spring WebSocket handler and STOMP messaging to eliminate the latency floor and reduce server polling load.
- BCrypt cost factor tuning: adjust hashing cost based on deployment hardware to balance security and performance.
- Minimax for larger boards: extend the AI using heuristic evaluation to support scalable board sizes.
- Public deployment: containerise and deploy to a cloud platform for external access.
- Puzzle mode: implement predefined board challenges using the existing evaluation logic.
- Rematch and game history: extend gameplay features with replay and session continuity.

References

- BLOCH, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
- CODD, E.F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- FIELDING, R.T., 2000. Architectural styles and the design of network-based software architectures. Doctoral dissertation, University of California, Irvine.
- FOWLER, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- GRIGORIK, I., 2013. *High performance browser networking*. O'Reilly Media, Sebastopol.
- KLEPPMANN, M. (2019). *Designing Data-Intensive Applications*. O'Reilly Media.
- MARTIN, R.C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- MOZILLA DEVELOPER NETWORK. (2024). Using the Fetch API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- OWASP Foundation. (2023). Password Storage Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- PIMENTEL, V., NICKERSON, B.G., 2012. Communicating and displaying real-time data with WebSocket. *IEEE Internet Computing*, 16(4): 45–53.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. (2024). PostgreSQL 16 Documentation. Retrieved from <https://www.postgresql.org/docs/16/index.html>
- REENSKAUG, T. (1979). *Models-Views-Controllers*. Xerox PARC Technical Note.
- RICHARDSON, C., 2019. *Microservices patterns: with examples in Java*. Manning Publications, Shelter Island.
- RUSSELL, S., & NORVIG, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- SILBERSCHATZ, A., KORTH, H.F., & SUDARSHAN, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
- SPRING SECURITY TEAM. (2024). Spring Security Reference Documentation. VMware Tanzu. Retrieved from <https://docs.spring.io/spring-security/reference/index.html>

THYMELEAF PROJECT. (2023). Tutorial: Using Thymeleaf. Retrieved from <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>

VMWARE TANZU. (2024). Spring Boot Reference Documentation. Retrieved from <https://docs.spring.io/spring-boot/documentation.html>

VON NEUMANN, J., & MORGENSTERN, O. (1944). Theory of Games and Economic Behavior. Princeton University Press.

ZERMELO, E., 1913. Über eine Anwendung der Mengenlehre auf die Theorie des Schachspiels. Proceedings of the Fifth International Congress of Mathematicians, 2: 501–504.

Appendix: Project Repository

The complete source code for this project is maintained in a Git repository. The final submission is based on the feature/security-and-ai branch, which contains the BCrypt upgrade and the Minimax AI implementation described in Chapters 10 and 11.

Repository: <https://github.com/Haze-Ez/spring-distributed-tictactoe>

Primary branch: [feature/security-and-ai](#)

The src/ directory structure is as follows:

```
src/main/java/org/example/tictactoe/
├── TictactoeApplication.java      Spring Boot entry point
├── AppUser.java                  JPA entity: user identity + stats
├── config/
│   └── SecurityConfig.java       Filter chain, BCrypt, user seeder
├── controller/
│   ├── AuthController.java       POST /register, POST /login
│   └── GameController.java       All /game/** endpoints
├── model/
│   └── Game.java                 JPA entity: board, history, state, AI
├── repository/
│   ├── AppUserRepository.java
│   └── GameRepository.java
├── service/
│   ├── DbUserDetailsService.java Spring Security UserDetailsService
│   └── GameService.java          Business logic, Minimax, stats
├── src/main/resources/templates/
│   ├── game.html                Board, polling JS, leaderboard
│   ├── games.html               Game list, open games
│   ├── mode.html                Mode + difficulty selection
│   ├── login.html / register.html
│   └── puzzle.html / Awaiting.html Feature placeholders
├── src/test/java/org/example/tictactoe/
│   ├── TictactoeApplicationTests.java 5 MockMvc integration tests
│   └── model/GameTest.java            8 unit tests for game logic
```

Figure 20 - Directory structure